

Adaptive Large Neighborhood Prioritized Search for Optimization in Answer Set Programming

Irumi Sugimori¹ Maki Miyashita¹ Hidetomo Nabeshima²

Torsten Schaub³ Takehide Soh¹ Naoyuki Tamura⁴ Mutsunori Banbara¹

¹Nagoya University ²University of Yamanashi

³Universität Potsdam ⁴Kobe University

September 9, 2026

Large Neighborhood Search (LNS) [Shaw,'98]

LNS is a metaheuristic that starts with an initial solution and then iteratively seeks better solutions by alternately **destroying** and **repairing** a current solution.

- 😊 LNS and its adaptive extension (ALNS) have been so far successfully applied in the areas of routing and scheduling problems.
- 😊 Since the **repair** operators can be implemented with **systematic solvers**, LNS has been shown to be highly compatible with
 - **ASP**
 - MIP [Fischetti+, '03; Danna+, '05]
 - CP [Shaw, '98; Dekker+, '18; Björdal+, '19, '20]

ALASPO: ASP-based ALNS [Eiter+, AAI'22, KR'22, AIJ'24]

Can improve the performance of ASP optimization on hard problems.

Large Neighborhood Prioritized Search (LNPS)

LNPS [Sugimori+, '24] is an extension of LNS that starts with an initial solution and then iteratively seeks better solutions by alternately destroying and **prioritized search** a current solution.

- Prioritized search is a systematic search method employing a customizable decision heuristic, such as VSIDS.
- 😊 LNPS enables flexible search without heavy reliance on destroy operators, as the undestroyed part is fixed heuristically rather than rigidly and can be reassigned (**variability**).
- 😊 LNPS guarantees optimal solutions (**optimality**).

heulingo [LNPS]: ASP-based LNPS [Sugimori+, '24]

- Achieves competitive performance compared to ALASPO.

LNS/LNPS Algorithm

Algorithm 1 LNS

Input: feasible solution x , LNS configuration γ

```
1:  $x^* \leftarrow x$ 
2: while stop criterion is not met do
3:    $x^t \leftarrow \text{repair}(\text{destroy}(x, \gamma), \gamma)$ 
4:   if  $\text{accept}(x^t, x)$  then
5:      $x \leftarrow x^t$ 
6:   end if
7:   if  $c(x^t) < c(x^*)$  then
8:      $x^* \leftarrow x^t$ 
9:   end if
10: end while
11: return  $x^*$ 
```

The *repair* operator can be typically implemented using greedy algorithms.

LNS/LNPS Algorithm

Algorithm 2 Declarative LNS

Input: feasible solution x , LNS configuration γ

```
1:  $x^* \leftarrow x$ 
2: while stop criterion is not met do
3:    $x^t \leftarrow \text{search}(\text{destroy}(x, \gamma), \gamma)$ 
4:   if  $\text{accept}(x^t, x)$  then
5:      $x \leftarrow x^t$ 
6:   end if
7:   if  $c(x^t) < c(x^*)$  then
8:      $x^* \leftarrow x^t$ 
9:   end if
10: end while
11: return  $x^*$ 
```

Declarative LNS uses systematic search instead of repair.

LNS/LNPS Algorithm

Algorithm 3 LNPS

Input: feasible solution x , LNPS configuration γ

```
1:  $x^* \leftarrow x$ 
2: while stop criterion is not met do
3:    $x^t \leftarrow \text{prioritized-search}(\text{destroy}(x, \gamma), \gamma)$ 
4:   if  $\text{accept}(x^t, x)$  then
5:      $x \leftarrow x^t$ 
6:   end if
7:   if  $c(x^t) < c(x^*)$  then
8:      $x^* \leftarrow x^t$ 
9:   end if
10: end while
11: return  $x^*$ 
```

Prioritized search can be implemented by *clingo*'s `#heuristic` directives.

Problem Issues

Issue 1: Sensitivity to destruction percentages

ASP-based LNS is highly sensitive to the **destruction percentage**.

- It must be sufficiently **large** to ensure that the neighborhood contains improved solutions.
- It must be sufficiently **small** to allow the ASP solver to explore the neighborhood efficiently.

Issue 2: A priori parameter determination

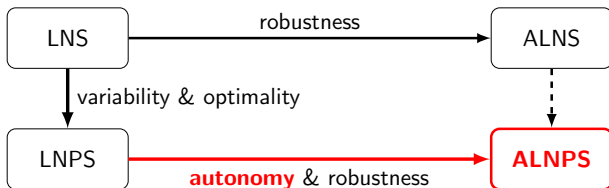
Determining optimal destruction levels a priori remains a significant computational challenge.

- Identifying optimal destruction percentages for course timetabling required approximately **2,500 CPU hours** [Sugimori+, '24].
- This computational overhead is a major barrier to applying declarative LNS methods to various optimization problems.

Our Proposal

Adaptive Large Neighborhood Prioritized Search (ALNPS)

- ALNPS extends LNPS with enhanced capabilities in **autonomy** and robustness.
- ALNPS alternates between **variability-guided destruction (VGD)** and prioritized search on the current solution.



What's autonomy of ALNPS?

VGD automatically determines the destruction percentage in each iteration based on **differences** between consecutive solutions.

Variability-Guided Destruction

Key Idea

We exploit **neighborhood variability of LNPS** to automatically determine the destruction percentage in each iteration.

- In LNS and ALNS, the undestroyed part remains **fixed** during repair.
 - The specified destruction percentage directly determines the amount of change introduced in each iteration.
- In LNPS, the undestroyed part is **preferred heuristically**.
 - It may be modified during reconstruction, causing the actual amount of change to differ from the specified destruction percentage.
- Therefore, the **difference** between consecutive solutions provides a simple way to **observe the actual change**.
- This difference can be used as feedback for determining the destruction percentage in subsequent iterations.

ALNPS Algorithm

Algorithm 4 ALNPS

Input: feasible solution x , ALNPS configuration Γ

```
1:  $x^* \leftarrow x$ 
2:  $\gamma \leftarrow \text{getInitialConfig}(\Gamma)$ 
3: while stop criterion is not met do
4:    $x^t, \text{stats} \leftarrow \text{prioritized-search}(\text{destroy}(x, \gamma), \gamma)$ 
5:   if  $\text{accept}(x^t, x)$  then
6:      $x \leftarrow x^t$ 
7:   end if
8:   if  $c(x^t) < c(x^*)$  then
9:      $x^* \leftarrow x^t$ 
10:  end if
11:   $\gamma \leftarrow \text{updateConfig}(\gamma, \Gamma, \text{stats})$ 
12: end while
13: return  $x^*$ 
```

- Line 2: The *getInitialConfig* function selects an initial configuration from the portfolio Γ , and assigns it to γ .
- Line 11: The *updateConfig* function updates γ for next iteration.

The updateConfig Function

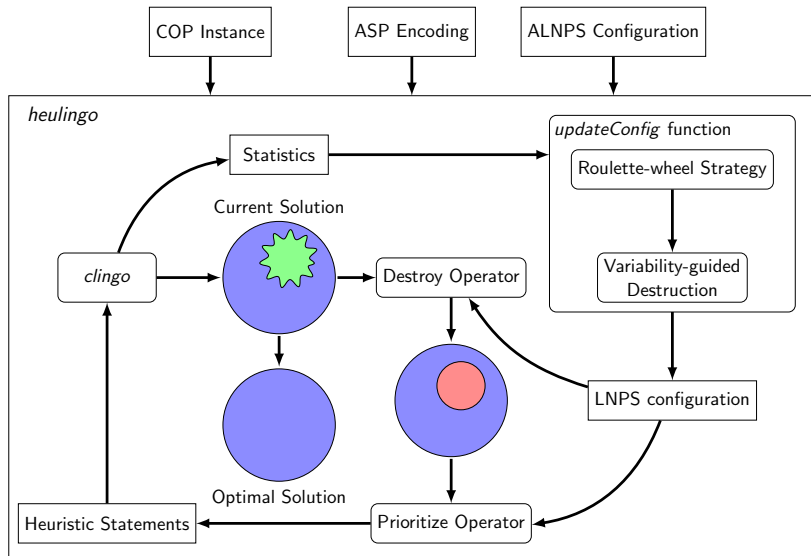
Algorithm 5 *updateConfig* function

Input: LNPS configuration γ , ALNPS configuration Γ , Statistics *stats*

```
1:  $\gamma' \leftarrow \text{selectConfig}(\gamma, \Gamma, \text{stats})$ 
2: if  $\gamma'$  is variability-guided destruction then
3:    $p \leftarrow \text{computePercentage}(\text{stats})$ 
4:    $\gamma' \leftarrow \text{updatePercentage}(\gamma', p)$ 
5: end if
6: return  $\gamma'$ 
```

- ALNPS first selects the next configuration γ' from the portfolio Γ .
- If γ' uses Variability-Guided Destruction, ALNPS computes a percentage p based on *stats*.
- The destruction percentage of γ' is then updated to p .

heulingo [ALNPS]: ASP-based ALNPS Implementation



Implementation of Variability-Guided Destruction

- The *heulingo* [ALNPS] solver begins with an arbitrary destruction percentage (0% by default).
- After each iteration, it determines the next percentage from the difference between the current solution x and the new solution x^t .
- To compute the difference, several **distance measures** can be used:
 - 1 Asymmetric set difference
 - 2 Hamming distance (i.e., Jaccard distance)
 - 3 Dice distance
 - 4 Overlap-based distance

Example of Variability-Guided Destruction (VGD)

VGD based on asymmetric set difference

Let x and x^t be the current solution and the new solution.

$$p = \frac{|x - x^t|}{|x|} \times 100$$

The resulting value p is used as the destruction percentage for the next iteration.

VGD example on timetabling

current solution x

	Mon	Tue	Wed	Thu	Fri
1	ENG	MATH	SCI	PE	ART
2	MATH	ENG	CHEM	CS	GEO
3	SCI	PE	MATH	ENG	SCI
4	PE	ART	MUSIC	MATH	CS
5	HIST	GEO	ENG	CHEM	HIST

new solution x^t

	Mon	Tue	Wed	Thu	Fri
1	ENG	MATH	SCI	PE	ART
2	MATH	CHEM	CHEM	CS	GEO
3	SCI	PE	GEO	ENG	MATH
4	PE	ART	MUSIC	MATH	CS
5	HIST	ENG	ENG	SCI	HIST

$$p = 5/25 * 100 = 20\%$$

Experiments on Timetabling

We evaluate ALNPS on 61 Course Timetabling instances, including the ITC competition instances [Gaspero+, '07].

- We compare
 - **clingo**-5.8.0
 - The best two configurations found in [Banbara+, '19].
 - **heulingo [LNPS]**
 - The best destruction percentage found in [Sugimori+, '24].
 - **heulingo [ALNPS]**
 - VG-Random: variability-guided random destruction (**Proposal**)
 - **ALASPO**¹
 - The default portfolio combined with 6 strategies [Eiter+, '22, '24]²
- **Time limit**: 1 hour per instance

¹<https://gitlab.tuwien.ac.at/kbs/BAI/alaspo>

²the uniform roulette-wheel strategy, the dynamic strategy, and the self-adaptive roulette-wheel strategy with $\alpha = \{0.2, 0.4, 0.6, 0.8\}$

Comparison of Obtained Bounds

Instance	<i>clingo</i> (ASP)		<i>heulingo</i> [LNPS] Random 6%			<i>heulingo</i> [ALNPS] VG-Random			<i>ALASPO</i> (ALNS) Portfolios		
	bb	usc	avg.	min.	max.	avg.	min.	max.	avg.	min.	max.
comp01	124	283	14.3	13	15	11.0	11	11	11.0	11	11
comp02	989	331	239.7	231	250	233.7	206	248	390.7	212	723
comp03	791	302	197.0	184	214	206.3	199	212	299.3	246	340
comp04	211	*49	*49.0	*49	*49	*49.0	*49	*49	51.5	49	57
comp05	2662	1940	928.7	782	1029	996.3	888	1094	1643.2	1310	2064
comp06	822	1025	209.0	195	222	212.7	204	219	381.3	238	547
comp07	924	1149	216.0	145	312	166.7	139	201	856.0	315	1157
				⋮							
DDS1	6536	7544	3820.0	3656	4031	3886.3	3848	3950	4825.0	4631	5041
DDS2	402	433	83.3	73	93	75.3	73	77	81.3	64	98
DDS3	22	391	23.3	22	24	24.7	24	26	22.0	22	22
DDS4	10486	3123	3177.3	3108	3237	1273.7	1267	1285	10234.3	9472	11629
DDS5	539	*76	*76.0	*76	*76	*76.0	*76	*76	141.0	89	216
DDS6	849	964	166.3	138	188	157.7	135	177	774.0	251	1130
DDS7	645	1485	541.3	502	610	113.3	105	119	681.2	129	943
#best averages	4	8	23			32			14		
Average ratio	1.796	1.000	0.592	0.571	0.614	0.497	0.462	0.532	0.902	0.680	1.173

Compare with *clingo* (baseline)

Our *heulingo* [ALNPS] improved the bounds of *clingo* by **50.3%** on average.

Comparison of Obtained Bounds

Instance	<i>clingo</i> (ASP)		<i>heulingo</i> [LNPS] Random 6%			<i>heulingo</i> [ALNPS] VG-Random			ALASPO (ALNS) Portfolios		
	bb	usc	avg.	min.	max.	avg.	min.	max.	avg.	min.	max.
comp01	124	283	14.3	13	15	11.0	11	11	11.0	11	11
comp02	989	331	239.7	231	250	233.7	206	248	390.7	212	723
comp03	791	302	197.0	184	214	206.3	199	212	299.3	246	340
comp04	211	* 49	* 49.0	* 49	* 49	* 49.0	* 49	* 49	51.5	49	57
comp05	2662	1940	928.7	782	1029	996.3	888	1094	1643.2	1310	2064
comp06	822	1025	209.0	195	222	212.7	204	219	381.3	238	547
comp07	924	1149	216.0	145	312	166.7	139	201	856.0	315	1157
				⋮							
DDS1	6536	7544	3820.0	3656	4031	3886.3	3848	3950	4825.0	4631	5041
DDS2	402	433	83.3	73	93	75.3	73	77	81.3	64	98
DDS3	22	391	23.3	22	24	24.7	24	26	22.0	22	22
DDS4	10486	3123	3177.3	3108	3237	1273.7	1267	1285	10234.3	9472	11629
DDS5	539	* 76	* 76.0	* 76	* 76	* 76.0	* 76	* 76	141.0	89	216
DDS6	849	964	166.3	138	188	157.7	135	177	774.0	251	1130
DDS7	645	1485	541.3	502	610	113.3	105	119	681.2	129	943
#best averages	4	8	23			32			14		
Average ratio	1.796	1.000	0.592	0.571	0.614	0.497	0.462	0.532	0.902	0.680	1.173

Compare with LNPS using manually tuned destruction percentage

Our *heulingo* [ALNPS] obtained improved bounds for many instances.

Comparison of Obtained Bounds

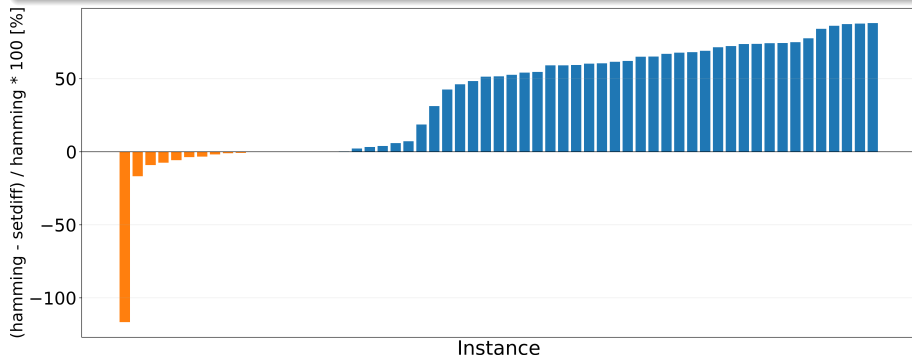
Instance	<i>clingo</i> (ASP)		<i>heulingo</i> [LNPS] Random 6%			<i>heulingo</i> [ALNPS] VG-Random			ALASPO (ALNS) Portfolios		
	bb	usc	avg.	min.	max.	avg.	min.	max.	avg.	min.	max.
comp01	124	283	14.3	13	15	11.0	11	11	11.0	11	11
comp02	989	331	239.7	231	250	233.7	206	248	390.7	212	723
comp03	791	302	197.0	184	214	206.3	199	212	299.3	246	340
comp04	211	*49	*49.0	*49	*49	*49.0	*49	*49	51.5	49	57
comp05	2662	1940	928.7	782	1029	996.3	888	1094	1643.2	1310	2064
comp06	822	1025	209.0	195	222	212.7	204	219	381.3	238	547
comp07	924	1149	216.0	145	312	166.7	139	201	856.0	315	1157
				⋮							
DDS1	6536	7544	3820.0	3656	4031	3886.3	3848	3950	4825.0	4631	5041
DDS2	402	433	83.3	73	93	75.3	73	77	81.3	64	98
DDS3	22	391	23.3	22	24	24.7	24	26	22.0	22	22
DDS4	10486	3123	3177.3	3108	3237	1273.7	1267	1285	10234.3	9472	11629
DDS5	539	*76	*76.0	*76	*76	*76.0	*76	*76	141.0	89	216
DDS6	849	964	166.3	138	188	157.7	135	177	774.0	251	1130
DDS7	645	1485	541.3	502	610	113.3	105	119	681.2	129	943
#best averages	4	8	23			32			14		
Average ratio	1.796	1.000	0.592	0.571	0.614	0.497	0.462	0.532	0.902	0.680	1.173

Compare with ALNS using pre-defined percentages

Our *heulingo* [ALNPS] obtained improved bounds for many instances.

Comparison of Different Distance Measures

We compared **set difference** $\frac{|x-x_t|}{|x|}$ with **Hamming distance** $\frac{|x-x_t|+|x_t-x|}{|x \cup x_t|}$.



- The plot shows the relative improvement in the objective value obtained by set difference compared with Hamming distance.
- Set difference outperforms Hamming distance on **42** instances, achieving improvements of more than **50%** on **31** instances.

Further Experiments

We carry out experiments on a challenging benchmark set [Eiter+,KR'22].

- The benchmark set:
 - 1 Traveling Salesperson Problem (TSP)
 - 2 Social Golfer Problem (SGP)
 - 3 Sudoku Puzzle Generation (SPG)
 - 4 Weighted Strategic Companies (WSC)
 - 5 Shift Design (SD)
 - 6 Partner Units Problem (PUP) [Eiter+,'24]
- We compare four solvers:
 - 1 *clingo*
 - 2 *heulingo* [LNPS] with the best percentages in [Sugimori+,'24].
 - 3 *heulingo* [ALNPS] (**Proposal**)
 - 4 *ALASPO* with the best portfolio in [Eiter+,'22,'24].

Comparison of the average ratios to clingo

- *clingo* = 1.000; lower is better.

	<i>heulingo</i> [LNPS]	<i>heulingo</i> [ALNPS]	<i>ALASPO</i> (ALNS)
TSP	0.650	0.649	0.716
SGP	0.744	0.879	0.794
SPG	0.664	0.658	0.658
WSC	0.915	0.933	0.970
PUP	0.981	0.981	0.982
SD*	1	0	7

Tuned

No tuning

heulingo [ALNPS] demonstrated that ALNPS can be competitive to LNPS and ALNS with tuned configurations.

Conclusion

We proposed adaptive Large Neighborhood Prioritized Search (ALNPS) for solving COPs, and presented an ASP-based implementation of ALNPS.

All source code is available from:

